Dyna Putt: Does Imagined Practice Help an Agent Learn a Mini-Golf Course?

Ali Seifeldin | COMP 765 -- Fall 2026

Abstract

Dyna reuses a learned model to make extra value updates between real actions. We test how much that saves in a small, fully inspectable mini-golf game, comparing tabular Q-learning with Dyna-Q at 5 and 20 planning updates per real shot on two deterministic courses. In the five-seed primary experiment, more planning means fewer real shots to an 80% rolling success threshold on both courses (5063, 1848, and 1607 shots on the harder course). A 100-seed study confirms the direction, shows sharply diminishing returns, and indicates that an ordering reversal in an earlier run was five-seed noise rather than an effect of its random-number protocol. A control then qualifies the headline. Every shot costs -1, so a zero-initialized Q table is optimistic. With non-optimistic initial values the planning advantage on the two-shot course disappears on the threshold metric, and on the five-shot course it shrinks from about 3.2x to 1.3-1.5x. A family of longer serpentine courses shows the other side of that optimism: with the lowest possible initial value most agents never solve an eight-shot course, while zero-initialized agents always do. How much planning appears to help therefore depends strongly on initialization, and initialization has to balance finding the goal against settling on a route.

1. Introduction

An agent that learns only from real interaction may need many trials before useful behavior emerges. A learned model offers another source of updates: after seeing how the world responds, the agent can replay that outcome internally. Sutton's Dyna architecture joins trial-and-error learning, model learning, and planning in one loop [1]. This project examines that loop in a mini-golf game small enough that every model entry can be read directly.

The research question is: *does Dyna-Q learn a course in fewer real shots than ordinary Q-learning, and does its advantage grow with the number of imagined updates?* Two hypotheses were fixed before the experiment. **H1:** Dyna-Q needs fewer real shots than Q-learning. **H2:** the advantage grows from 5 to 20 updates per shot. Two more came later, from auditing the first results, so they are exploratory. **H3:** part of the advantage comes from the optimistic zero initial Q table rather than from planning over routes as such. **H4:** once that optimism is reduced, the remaining benefit grows with route length.

The contribution is modest and empirical. Beyond a working adaptation of Dyna-Q, we show in this setting how quickly the benefit saturates, that a five-seed ordering reversal was sampling noise, and that initialization accounts for most of the measured gain on short routes while also deciding whether long routes are found at all. These later results changed the conclusion, because the original comparison alone would have credited planning with the whole gain.

2. Related Work

2.1 Dyna and where to spend planning

Dyna interleaves acting, model learning, and planning over model-generated experience. Sutton's 1990 paper presents the Dyna-PI and Dyna-Q architectures as forms of incremental approximate dynamic programming [1], and a short paper followed [2]. The usual Dyna-Q samples previously observed pairs uniformly [8], which raises the question of whether that is a good use of updates. Prioritized sweeping keeps a priority queue and spends its backups where the largest possible value change lies [3], and Peng and Williams also studied how to make Dyna's computation more efficient [4]. Our uniform sampler is the simple starting point, and Section 6 returns to this idea.

2.2 Planning, replay, and initial values

Experience replay reuses stored transitions [5]. In a deterministic tabular task with one stored outcome per pair, a Dyna-Q planning update is close to a replay update, and the relationship between the two has been analyzed directly [6, 7]. Dyna Putt therefore tests model reuse at its most favorable, with no model error at all. Initial values matter for a separate reason. Sutton and Barto present optimistic initial values as a simple way to drive exploration [8]. Koenig and Simmons show that under an action-penalty reward like ours, zero initial values make reaching the goal tractable for greedy Q-learning with learning rate 1, where a goal-reward form with zero values can need exponentially many actions [9]. So a zero table is a sensible default for finding the hole, but it is not neutral. Our control asks what that optimism costs once the hole has been found.

2.3 Learned models at scale

Later methods replace the table with learned function approximators, which makes model error part of the problem. PETS uses an ensemble of probabilistic neural dynamics models and propagates sampled trajectories during planning [10]. MBPO keeps Dyna's structure, generating short model rollouts from real states to train a policy, and limits rollout length because errors compound [11]. MuZero uses tree search over a learned model without being given the rules [12]. TD-MPC combines latent dynamics with short-horizon trajectory optimization and a learned terminal value [13], and TD-MPC2 scales that design across many continuous-control tasks [14]. DreamerV3 learns behavior from imagined latent rollouts across many domains [15]. Discrete Codebook World Models use discrete latent codes with decision-time planning [16]; those codes are learned, whereas our discrete states are directly observed cells. Dyna Putt implements none of these. It shares only the idea of spending computation on a model to save real experience.

2.4 Connection to the course

The project sits in the course's model-based reinforcement learning and modern world-model topics, and the course's first assignment offers two direct comparisons. That assignment profiled DreamerV3, which trains an actor and critic on short imagined latent trajectories and then acts without searching. Dyna-Q has the same shape, learning from imagined experience and then acting directly from its values, but it imagines by reading a table rather than by predicting with a learned latent model. The assignment also fitted a compact cart-pole dynamics model from 7,680 simulator transitions and used it in a 50 ms receding-horizon controller with an LQR terminal value.

That controller shows the other main use of a model. It consults the model at decision time, comparing candidate forces at every step, much as PETS and TD-MPC do, and its terminal value plays the role of TD-MPC's learned one [13]. Dyna plans in the background instead: the model improves stored values, and the resulting greedy policy needs no model when it acts. The cart-pole model had to be checked for prediction error on held-out data, whereas here the model is exact once a pair is seen. Dyna Putt thus isolates the planning question by removing representation learning and model error.

3. Method

3.1 Environment

A state is the grid cell where the ball rests between shots. Each of the 16 actions combines one of eight directions with 2 or 4 movement ticks. The ball moves one cell per tick until power runs out or it reaches the hole. Walls reflect the blocked component of its direction and the bumper reverses it. Every shot gives reward -1. Sinking is terminal. An episode also ends after 20 shots, but that is a truncation: the update still bootstraps from the resting cell, so the limit is not learned as a goal. The physics are deterministic, so each state-action pair has exactly one outcome.

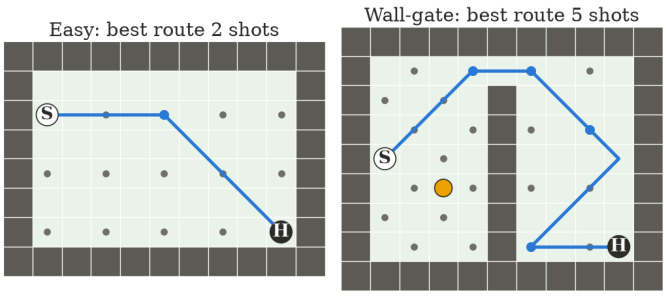


Figure 1: The two courses, with one shortest route found by exhaustive search. Gray dots are the resting cells reachable from the start: 15 of 54 cells on Easy and 23 of 56 on Wall-gate. S is the start, H the hole, and the orange circle the bumper.

The Easy layout has 54 traversable cells and Wall-gate has 56, but because shots move two or four cells only 15 and 23 are reachable from the start (Figure 1). Breadth-first search over all transitions gives shortest routes of 2 and 5 shots. The effective problems are therefore small: 224 and 352 reachable non-terminal state-action pairs. Wall-gate is harder mainly because the route is longer and must go around the wall.

3.2 The target paper and how this project differs

Sutton's paper describes an agent with three coupled processes. It acts in the world, it updates a world model from what it sees, and between real steps it runs k hypothetical steps in which the model stands in for the world and the same learning rule is applied [1]. Planning is thus not a separate search stage. It is the learning algorithm run on remembered experience, which makes it incremental and interruptible. Dyna-PI builds this around policy iteration with separate evaluation and policy structures. Dyna-Q replaces both with Q-values, which is simpler and tolerates a difference between how hypothetical actions are chosen and the current policy.

The experiments use small grid mazes. A navigation task compares learning with and without planning (for example k=0 and k=100). Changing-world tasks then block the known path or open a shortcut, comparing Dyna-PI, Dyna-Q, and Dyna-Q+. The last adds an exploration bonus that grows with the square root of the time since an action was tried, and it is what finds the shortcut. Two of the paper's stated limitations matter here. Search control, the choice of which hypothetical experiences to run, is kept trivial because the tasks are small, and reward is non-zero only when a path ends [1].

	Sutton (1990) [1]	This project
Task	Grid maze, one cell per step	Mini-golf, multi-cell shots
Reward	Non-zero only at path end	-1 on every shot
Zero initial Q	Not optimistic	Optimistic everywhere
Model	Learned table	Learned table
Search control	Random	Uniform over seen pairs
Tested change	Blocks and shortcuts	Budget, initial Q, route length

Table 1: The target paper and this adaptation. We do not reproduce Sutton's experiments or Dyna-PI, and we do not test changing worlds.

The reward row turned out to be the important one. With reward only at the goal, a zero table makes no promise about unexplored actions. With a cost on every shot, the same table promises more than any action can deliver.

3.3 Q-learning and Dyna-Q

For a transition (s, a, r, s') with d=1 only on a sink, both agents apply

$$Q(s,a) \leftarrow Q(s,a) + \alpha[\, r + \gamma(1-d)\max_{a'} Q(s',a') - Q(s,a)\,]$$

with α=0.25 and γ=0.95. Actions are ε-greedy, with ε starting at 1.0, multiplied by 0.992 after each episode, and floored at 0.03. Random ties among best actions are broken at random in training. Dyna-Q also stores the latest (r, s', d) for each observed pair. After each real update it draws k known pairs uniformly with replacement and applies the same update to their stored outcomes. Setting k=0 gives Q-learning. Planning never reveals an untried pair; it only spreads what was already seen. Each real shot costs 1+k updates, so k=20 does 21 times the update work of k=0. Algorithm 1 gives the loop.

Algorithm 1: Dyna-Q as implemented
Initialize $Q(s,a) = q_0$ for all s, a; model M empty
For each episode, until a sink or 20 shots:
1. Choose a from s, ε-greedy on Q
2. Take the shot; observe r, s', and d
3. Update Q(s,a) from (r, s', d)
4. Store M(s,a) = (r, s', d)
5. Repeat k times: draw a stored (x,b) uniformly, read (r, y, e) = M(x,b), and update Q(x,b) from it
6. Set s = s'

After the episode: $\varepsilon = \max(0.03, 0.992\varepsilon)$

3.4 Why planning should help, and a second reason

The standard argument is propagation. Value starts at the sink and moves back one transition per backup. A route of L shots needs at least L backups in the right order, and without planning each backup requires physically replaying that transition. Planning performs them for free once the transitions are stored, so the benefit should grow with L.

A second effect comes from initialization. All returns here are negative: the optimal start values are $-(1-\gamma^L)/(1-\gamma)$, or -1.95 for L=2 and -4.52 for L=5, so Q=0 overestimates every pair. A tried action therefore looks worse than any untried one, and even a correct action must wait for its rivals' values to fall. With a target near -1, a value after n updates is about $-(1-0.75^n)$, which needs about eight updates to reach -0.9. Across 224 to 352 pairs that is thousands of updates. Without planning each one costs a real shot; with k=20 they are nearly free. So planning can speed learning simply by draining optimism, which says little about planning over routes. H3 separates these effects by changing the initial value.

Optimism also has a benefit. It sends the agent to untried actions without relying on luck, which is why zero values make goal-reaching tractable in Koenig and Simmons's analysis [9]. With very low initial values, anything already tried looks better than anything untried, so only ε drives exploration. Planning cannot repair this, because it only reuses seen pairs. We therefore expect low initial values to do well on short routes and to fail when the hole is unlikely to be found by chance before ε decays.

4. Experimental Setup

Primary experiment. Three planning settings (k = 0, 5, 20) on two courses with seeds 11, 23, 37, 41, and 59 and 350 training episodes each, giving 10,500 episode records. Action selection and planning draw from separate random streams, so planning draws do not directly change exploration draws. Streams can still diverge once the agents' behavior differs. Each course-setting pair starts from a new Q table and model.

Metrics. The question concerns real shots, so we report training shots up to the *threshold*, the first episode that ends a 20-episode window with at least 16 sinks, along with that episode number. Training success includes exploration, so we also evaluate the greedy policy after every episode. Physics and greedy tie-breaking are deterministic, so one rollout gives the exact outcome. *Greedy-stable* is the first episode after which all later checkpoints sink, counted only if at least 50 remain. We also report success over the checkpoints after episodes 301-350.

Extended studies. Five seeds cannot resolve small differences, so we made 6,600 further runs on 100 new seeds (1000-1099), kept apart from the primary data and never pooled with it. The *budget* study uses k = 0, 1, 2, 5, 10, 20, 50. The *stream* study repeats k = 5 and 20 with a single shared random stream, the protocol of an earlier exploratory run; k=0 makes no planning draws, so it needs no repeat. The *initialization* study repeats k = 0, 5, 20 with initial Q of -5, near the Wall-gate start value, and -20, the lowest possible value $-1/(1-\gamma)$. These values were chosen from that reasoning rather than tuned. Intervals are 95% over seeds: normal approximations in figures and paired bootstrap for differences.

Route-length study. Easy and Wall-gate differ in many ways, so we built four serpentine courses by adding walls under one rule (Figure 2). Their shortest routes are 3, 4, 7, and 8 shots. Each runs k = 0, 5, 20 at all three initial values for 800 episodes, because zero planning needs longer on the largest course. Course size and wall count grow along with route length, so this family controls route length only loosely.

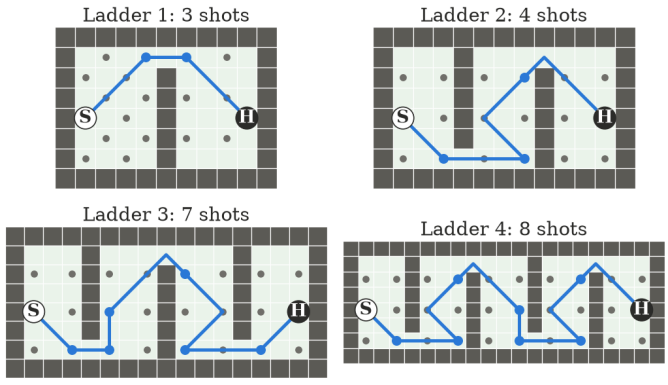


Figure 2: The serpentine ladder courses, drawn as in Figure 1. Reachable cells number 19, 18, 24, and 30.

5. Results

5.1 Primary experiment

Course	k	Threshold episode	Real shots	Greedy-stable	Greedy success
Easy	0	79.0 ± 9.9	1136 ± 115	169.8	1.000
Easy	5	61.0 ± 4.6	827 ± 54	57.6	1.000
Easy	20	49.2 ± 7.9	713 ± 132	48.6	1.000
Wall-gate	0	284.8 ± 18.4	5063 ± 231	none (0/5)	0.400
Wall-gate	5	103.8 ± 8.1	1848 ± 112	129.2	1.000
Wall-gate	20	89.8 ± 4.7	1607 ± 59	74.6	1.000

Table 2: Primary experiment, five seeds (mean ± sample SD). Greedy success covers checkpoints after episodes 301-350.

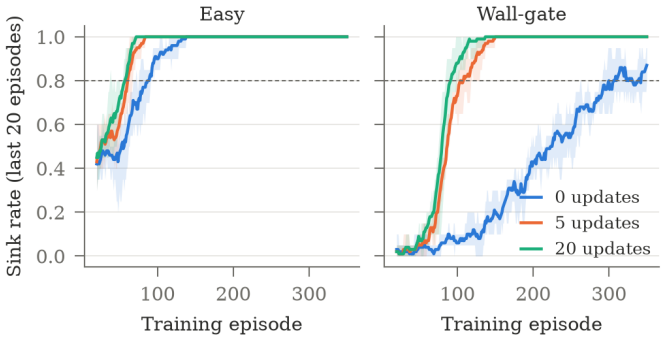


Figure 3: Primary experiment. Lines show the mean 20-episode training sink rate over five seeds; bands show the range across seeds. The dashed line is the 80% threshold.

Table 2 supports H1 and H2 in this run. All 30 runs reach the threshold, and on both courses k=0 is slower than k=5, which is slower than k=20, in all five paired seeds. On Wall-gate the agents need 5063, 1848, and 1607 real shots. On Easy they need 1136, 827, and 713. Every condition ends with a shortest-route policy when it is stable: 2 shots on Easy and 5 on Wall-gate.

The zero-planning Wall-gate result needs care. Its 0.400 is 100 of 250 checkpoints, but it does not mean two seeds solved the course and three failed. Each of the five seeds sinks in 17 to 24 of its last 50 checkpoints, and none is stable. The greedy policy is still flipping at episode 350, even though the threshold, reached near episode 285 under exploration, suggests success. This is why training success alone is a poor measure of a learned policy.

5.2 More seeds and more budgets

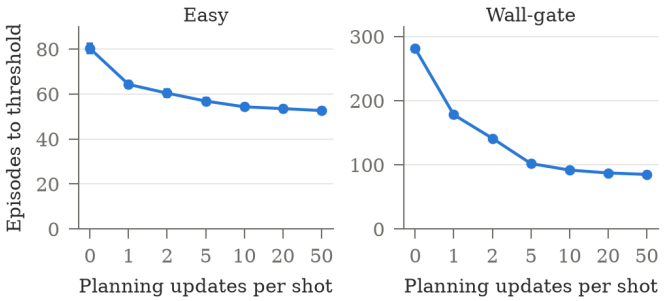


Figure 4: Budget study, 100 seeds, mean with 95% interval. Note the different vertical scales and the non-uniform horizontal spacing of budgets.

With 100 seeds the ordering of H2 holds on average, but the size of the effect changes (Figure 4). On Easy the gap between k=5 and k=20 is 3.4 episodes (95% CI 2.1 to 4.7), compared with 11.8 in the primary run, and k=20 is faster in only 64 of 100 paired seeds, with 7 ties. The primary run's five-of-five result was favorable. On Wall-gate the gap is 14.6 episodes (13.0 to 16.3), with k=20 faster in 99 of 100 seeds.

Returns diminish quickly. One update per shot delivers over half of the whole improvement available up to k=50: 16.0 of 27.6 episodes on Easy and 103.2 of 196.3 on Wall-gate. Moving from k=20 to k=50 saves only 0.8 and 2.3 episodes. The trade in computation is visible too. On Wall-gate, k=20 uses 3.2 times fewer real shots than k=0 (1573 against 5007) but 6.6 times more total updates, while k=1 uses 1.6 times fewer shots for 1.25 times the updates.

5.3 The earlier ordering reversal was noise

An earlier exploratory run, with planning and exploration sharing one random stream, had k=5 ahead of k=20 on Easy (50.4 against 54.4 episodes). We regenerated that archive exactly by restoring the shared stream, so the protocol difference is confirmed, and it is tempting to credit the protocol for the reversal. The stream study does not support that. With 100 seeds the shared stream gives 56.4 and 52.7 episodes on Easy, nearly matching 56.9 and 53.5 with separate streams, and k=20 stays ahead under both. Resampling five seeds from these runs puts k=5 ahead of k=20 in 12% (separate) to 21% (shared) of samples. The reversal is well explained by chance. Separate streams remain the cleaner design, but we find no sign that they changed the answer.

5.4 Initial values change the conclusion

Course	Initial Q	k=0	k=5	k=20	Stable k=0 / k=20
Easy	0	80.3	56.9	53.5	173.0 / 49.0
Easy	-5	28.6	30.1	29.0	4.1 / 2.5
Easy	-20	29.5	31.0	29.6	3.5 / 2.3
Wall-gate	0	281.3	101.9	87.3	none / 72.3
Wall-gate	-5	112.7	79.3	75.1	93.2 / 31.8
Wall-gate	-20	122.7	94.6	92.4	94.6 / 53.0

Table 3: Initialization study, 100 seeds. Middle columns give mean episodes to threshold; the last gives the mean greedy-stable episode for k=0 and k=20.

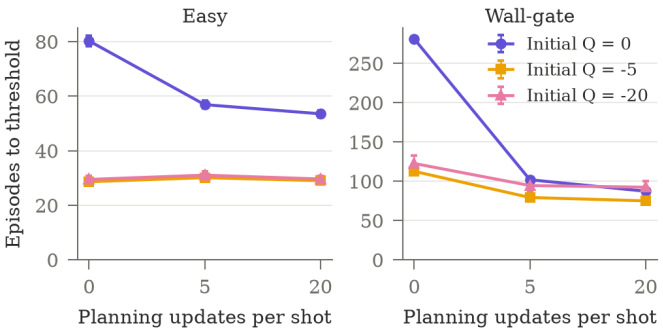


Figure 5: Episodes to threshold by planning budget and initial Q, 100 seeds, mean with 95% interval. Vertical scales differ.

Table 3 and Figure 5 support H3. On Easy, the k=0 minus k=20 threshold difference is 26.7 episodes (24.7 to 28.6) at initial Q=0, but -0.4 (-1.7 to 0.9) at -5 and -0.1 (-1.2 to 0.9) at -20. The planning advantage disappears, and plain Q-learning with a non-optimistic start reaches the threshold in about 29 episodes, far sooner than any zero-initialized Dyna agent (53).

The threshold is partly floored here by exploration: ε is still near 0.8 at episode 30, while the greedy policy is stable by about episode 4 with or without planning. On this course there is little left for planning to do.

On Wall-gate the effect shrinks but survives. The difference falls from 194.0 episodes (190.4 to 197.6) to 37.6 (35.3 to 39.8) at -5 and 30.2 (25.7 to 35.0) at -20, so the speedup drops from 3.2x to 1.5x and 1.3x. Greedy stability still improves substantially, from about 93 to 32 episodes at -5 and from 95 to 53 at -20. Zero-planning Q-learning with initial Q=-5 (112.7 episodes) comes close to zero-initialized k=5 (101.9) and needs no planning at all.

5.5 Route length and the two faces of optimism

Best route	Initial Q	Threshold k=0	Threshold k=20	Speedup	Seeds at threshold k=0 / k=20
3	0	161.2	70.8	2.3x	100 / 100
3	-5	67.1	52.9	1.3x	100 / 100
3	-20	62.5	54.5	1.1x	100 / 100
4	0	124.9	63.7	2.0x	100 / 100
4	-5	64.8	56.2	1.2x	100 / 100
4	-20	65.8	57.4	1.1x	100 / 100
7	0	310.7	99.3	3.1x	100 / 100
7	-5	147.2	87.4	1.7x	100 / 100
7	-20	349.1	189.3	n/a	90 / 98
8	0	445.2	121.2	3.7x	100 / 100
8	-5	223.9	108.6	2.1x	100 / 100
8	-20	573.5	332.4	n/a	6 / 32

Table 4: Route-length study, 100 seeds, 800 episodes. Threshold means use only seeds that reached it, so no speedup is given where some seeds failed.

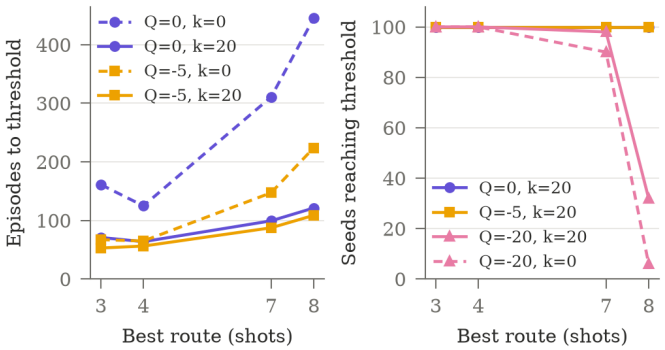


Figure 6: Route-length study. Left: mean episodes to threshold (dashed k=0, solid k=20). Right: seeds out of 100 that reach the threshold. Q=0 and Q=-5 reach it in every seed, so their lines coincide at 100.

H4 receives partial support (Table 4, Figure 6). At initial Q=-5, where every seed succeeds, the threshold speedup from k=20 is 1.3x, 1.2x, 1.7x, and 2.1x for routes of 3, 4, 7, and 8 shots, and episodes saved rise from 14.2 and 8.6 to 59.8 and 115.3. The trend is not strictly monotonic, and on the greedy-stable metric the speedup stays near 3x at every length (3.1x, 3.4x, 3.0x, 3.1x). So in absolute terms planning saves more on longer routes, but whether its relative benefit grows depends on the metric. At Q=0 speedups are larger throughout (2.0x to 3.7x), again reflecting the optimism cost.

The longest routes reveal the opposite risk. At the lowest initial value, -20, only 6 of 100 seeds at k=0 and 32 at k=20 ever reach the threshold on the 8-shot course, against all 100 at Q=0 and Q=-5. Planning raises the count, but k=5 and k=20 both stop at 32, consistent with planning being unable to help agents that never find the hole. Among the values tried, Q=-5 is fastest across the ladder at k=20 (52.9, 56.2, 87.4, and 108.6 episodes) and never fails. It is pessimistic near the hole yet still above the 8-shot start value of -6.73, so it keeps some drive to explore on long routes.

6. Discussion

H1 and H2 hold as stated, but they were measured under a condition that flatters planning. The pattern fits the two mechanisms in Section 3.4. Draining optimism is available on every course, and reducing the optimism removes much of the gain. Propagation should matter more on longer routes, and the absolute savings do grow there. This is an interpretation: the studies change the initial value and the course but do not measure either mechanism directly.

The initial value is best seen as a trade-off rather than a flaw. Zero costs many real shots on short routes but found the hole in every run here; -20 settles quickly on short routes and can fail on long ones. Planning interacts with both sides, since it cheapens the cost of optimism but cannot replace exploration. This also explains why the result could not have appeared in the same way in Sutton's setting: with reward only at the goal, zero values are not optimistic. A reader comparing Dyna variants across tasks should therefore check the reward form and initial values before attributing a gain to planning. A fair baseline for a model-based method is a well-initialized model-free agent.

The results also show how a small sample can mislead in two directions at once. Five seeds overstated the 5-versus-20 gap on Easy in the primary run and reversed it in the archived run. A confident story could have been told about either. All 6,600 extended runs take about two minutes on a laptop, so the small sample was never a real constraint here.

The main proposed improvement follows from Figure 4 and from Sutton's own note that search control was left trivial. Uniform sampling often revisits pairs whose values are no longer changing, which may be one reason returns saturate. Prioritized sweeping [3] would direct updates toward predecessors of pairs whose values just changed. We predict it would match k=20 with far fewer updates, with the saving growing with route length. A second idea is to pair a moderate initial value with a Dyna-Q+ style bonus [1], separating the exploration that optimism provides from the value estimates it distorts. We have implemented neither.

7. Limitations

This is a deliberately favorable setting for tabular planning: exact state, deterministic transitions, and a model that is correct once a pair is seen. It says nothing about model error, which is the central difficulty for the methods in Section 2.3, nor about the changing worlds Sutton tested. The six courses are hand-built, and in the ladder family route length moves together with size and wall count. All hyperparameters were fixed, so the ε schedule and α may interact with planning and initial value; the failures at -20 in particular depend on how fast ε decays and on the episode budget. Only three initial values were tried, and they were added after seeing the first results, so H3 and H4 need confirmation on new layouts. Threshold values depend on the 80% and 20-episode choices and are limited by ε when learning is fast. Intervals describe seed variation on these courses, not variation across tasks. Finally, we count real shots and updates but not wall-clock time.

8. Conclusion

In this game, Dyna-Q reaches reliable play in fewer real shots than Q-learning under the original configuration, and more planning helps with sharply diminishing returns. The stronger finding concerns attribution. Much of that benefit depends on an optimistic value table: none remains on the threshold metric for the two-shot course, and a smaller but clear benefit remains on the five-shot course. Removing optimism entirely is not the answer either, because long routes then often go unsolved. An apparent reversal between 5 and 20 updates turned out to be sampling noise. Relative to the starting assumptions, the main changes are that a zero table is not a neutral baseline and that a tidy five-seed result can mislead. Open questions are whether prioritized updates recover the benefit at lower cost, whether an explicit exploration bonus can replace optimism, and how these effects change once the model can be wrong.

Reflection. I began this project expecting a simple story in which more imagined practice always meant faster learning. What taught me most was how often a plausible story turned out to be wrong. When the first run showed five updates beating twenty on the easy course, I wrote explanations for it, such as diminishing returns on a simple layout. A rerun with separate random streams reversed the ordering, and it was tempting to credit the protocol change. One hundred seeds showed that neither reading was needed, because the reversal was ordinary noise. The biggest surprise was the initial value. I had treated a zero table as a neutral default, and did not expect one constant to account for so much of what I had attributed to planning, or for the opposite choice to leave long courses unsolved. The assumption that changed most was about baselines: a comparison is only as fair as the weaker agent's setup. In practice I would now run many seeds before interpreting an ordering, check that metrics match the question, since I had asked about shots while counting episodes, and run a demo rather than only reading its code.

Reproducibility. Code and data are at github.com/Bakalala/COMP765 in `dynaputt/`. `run_experiments.py` regenerates the primary data and `run_extended.py` the extended studies. A browser demo lets a visitor take shots and watch the agent; it trains its own illustrative agent (700 episodes, one random stream) and is not a measured checkpoint. Its transitions match the Python environment on all 1,728 non-terminal state-action pairs of Easy and Wall-gate.

References

[1] R. S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. *Proc. ICML*, pp. 216-224, 1990. DOI.

[2] R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *SIGART Bulletin* 2(4):160-163, 1991. DOI.

[3] A. W. Moore and C. G. Atkeson. Prioritized sweeping: Reinforcement learning with less data and less time. *Machine Learning* 13(1):103-130, 1993. DOI.

[4] J. Peng and R. J. Williams. Efficient learning and planning within the Dyna framework. *Adaptive Behavior* 1(4):437-454, 1993. DOI.

[5] L.-J. Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning* 8(3-4):293-321, 1992. DOI.

[6] H. van Seijen and R. S. Sutton. A deeper look at planning as learning from replay. *Proc. ICML*, PMLR 37:2314-2322, 2015. Paper.

[7] H. van Hasselt, M. Hessel, and J. Aslanides. When to use parametric models in reinforcement learning? *NeurIPS* 32, 2019. Paper.

[8] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed., Sections 2.6, 8.2, and 8.4. MIT Press, 2018. Book.

[9] S. Koenig and R. G. Simmons. The effect of representation and knowledge on goal-directed exploration with reinforcement-learning algorithms. *Machine Learning* 22(1-3):227-250, 1996. DOI.

[10] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *NeurIPS* 31, 2018. Paper.

[11] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. *NeurIPS* 32, 2019. Paper.

[12] J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature* 588(7839):604-609, 2020. DOI.

[13] N. Hansen, H. Su, and X. Wang. Temporal difference learning for model predictive control. *Proc. ICML*, PMLR 162:8387-8406, 2022. Paper.

[14] N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. *ICLR*, 2024. Paper.

[15] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse control tasks through world models. *Nature* 640(8059):647-653, 2025. DOI.

[16] A. Scannell, M. Nakhaei, K. Kujanpää, Y. Zhao, K. S. Luck, A. Solin, and J. Pajarinen. Discrete codebook world models for continuous control. *ICLR*, 2025. Paper.